

WEARABLE HEALTH MONITORING SYSTEM

PROJECT REPORT

Submitted By

PRIYADHARSHAN L 2023105055

AKHSHAIYA S 2023105507

VIGNESH B K 2023105509

KRISHNASHREE D 2023105539

**BACHELOR OF ENGINEERING
IN
ELECTRONICS AND COMMUNICATION**



COLLEGE OF ENGINEERING, GUINDY

**DEPARTMENT OF ELECTRONICS AND
COMMUNICATION ENGINEERING**

ANNA UNIVERSITY, CHENNAI

<i>S.No</i>	<i>Table of Contents</i>	<i>Page No.</i>
1	Abstract	3
2	Problem Statement	3
3	Feasible Solution	3
4	Objectives	3
5	Components Required	4
6	Schematic Diagram	4
7	Sensor Testing and Calibration	5
8	Software Implementation	6
9	Code	7
10	Working	10
11	Output	12
12	Advantages	13
13	Future Scope	13
14	Conclusion	14

WEARABLE HEALTH MONITORING SYSTEM

ABSTRACT:

The Wearable Health Monitoring System provides real-time tracking of vital health parameters, including heart rate, SpO₂ levels, body temperature, UV exposure, and activity levels. Traditional monitoring methods are often inconvenient, expensive, and require frequent medical visits, whereas this system offers a portable, cost-effective, and user-friendly solution.

The device integrates an ESP32 microcontroller with multiple sensors, including a UV sensor to track sunlight exposure between 6 AM and 6 PM, ensuring users take necessary precautions. Data is transmitted via Wi-Fi and Bluetooth to the Blynk IoT platform, enabling real-time access through both mobile and desktop applications. Additionally, customized alerts notify users of abnormal heart rates or high UV exposure, prompting timely action. Powered by a rechargeable battery, the system ensures continuous health monitoring, bridging the gap between traditional health tracking and modern IoT-driven solutions.

PROBLEM STATEMENT:

The problem is the absence of an effective system for real-time monitoring of vital health signs, which limits early detection, preventive care, and user empowerment. Many individuals overlook subtle health changes, which can lead to serious issues if not detected early. Traditional health monitoring methods are often inconvenient, expensive, and require frequent healthcare visits. Furthermore, existing wearable devices lack comprehensive tracking features, particularly for vital signs like UV exposure. There is a need for an affordable, user-friendly system that provides real-time updates on key health metrics, empowering users to manage their health proactively.

FEASIBLE SOLUTION:

The goal of a health monitoring system is to track important health signs, like pulse rate, oxygen levels, temperature, UV exposure and activity in real time. Along with existing sensors, we are adding a UV sensor to regulate tracking of exposure to sunlight. Moreover, we are using Blynk IOT, that can be accessed with an ESP32 which has a Bluetooth and a Wi-Fi module to display data in our mobile phones as well (making it easy for the users to access the data).

OBJECTIVES:

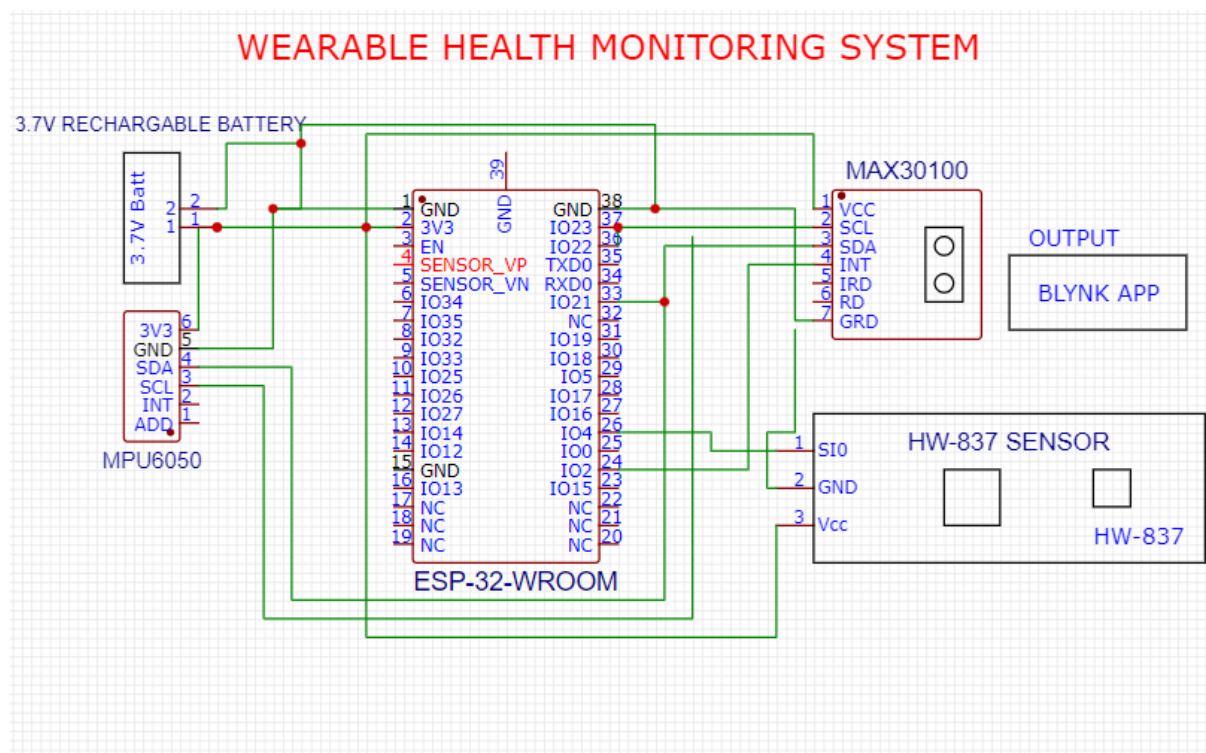
The Wearable Health Monitoring System should provide real-time health insights and proactive safety recommendations. It must continuously track heart rate, SpO₂ levels, body temperature, and UV exposure to help users make informed health decisions. The system should alert users when their pulse rate is too high, ensuring timely awareness of potential health risks. Additionally, it must monitor UV index levels between 6 AM and 6 PM, the most significant period for UV exposure, and provide precautionary suggestions such as using sunscreen or wearing a hat. Integrated with the Blynk IoT platform, the system should enable users to access their health data remotely via both mobile and desktop applications. Designed for portability, the wearable device must be powered by a rechargeable battery,

ensuring continuous health monitoring without dependency on external power sources. This system should empower users with real-time data and smart recommendations for a healthier and safer lifestyle.

COMPONENTS REQUIRED:

1. **ESP32 Microcontroller** – Handles data processing and Wi-Fi communication.
2. **MAX30100 Pulse Oximeter** – Measures pulse rate and blood oxygen levels.
3. **MPU6050 Accelerometer & Gyroscope** – Tracks motion and step count.
4. **HW-837 UV Sensor** – Detects UV radiation levels.
5. **Battery (Li-ion 3.7V)** – Powers the wearable device.
6. **Resistors and Wires** – For circuit connections and stability.
7. **Blynk IoT Platform** – For real-time data visualization and storage.
8. **Libraries required:**
 - MAX 30100_Pulseoximeter
 - Adafruit_MPU6050
 - Blynk
 - WiFi
 - NTPClient
 - Wire.h
 - Adafruit_sensor
9. **Tools:**
 - VS Code (with extensions installed) or Arduino ID
 - Blynk IOT application

SCHEMATIC DIAGRAM:



SENSOR TESTING AND CALIBRATION:

Before integrating the sensors with Blynk, each module was individually tested using Arduino Uno to verify proper functioning.

1. Testing Individual Sensors

MAX30100 Pulse Oximeter:

Heart rate and SpO₂ values were tested using a sample Arduino code. The sensor required finger placement adjustments for stable readings.

MPU6050 Accelerometer: Raw acceleration and gyroscope values were verified using the I2C scanner to ensure proper connection. With the help of a threshold value, we counted the number of steps taken by the user.

HW-837 UV Sensor: Readings were taken under different light conditions to test UV detection accuracy. This involved converting the sensor voltage taken in by the

2. Calibration & Debugging

We compared the raw data that we got from these sensors to the data available in the Serial debugging was used to monitor sensor outputs in real time before sending data to Blynk.

3. Codes used for Testing:

1. MAX30100 - Heart Rate & SpO₂ Sensor

```
#include <Wire.h>
#include "MAX30100_PulseOximeter.h"

PulseOximeter pox;
void onBeatDetected() { Serial.println("Beat!"); }

void setup() {
  Serial.begin(9600);
  if (!pox.begin()) while (1);
  pox.setOnBeatDetectedCallback(onBeatDetected);
}

void loop() {
  pox.update();
  Serial.print("HR: "); Serial.print(pox.getHeartRate());
  Serial.print(" bpm | SpO2: "); Serial.println(pox.getSpO2());
  delay(1000);
}
```

2. MPU6050 - Accelerometer

```
#include <Wire.h>
```

```

#include <MPU6050.h>

MPU6050 mpu;

void setup() {
  Serial.begin(9600);
  Wire.begin();
  if (!mpu.testConnection()) while (1);
}

void loop() {
  int16_t ax, ay, az;
  mpu.getAcceleration(&ax, &ay, &az);
  Serial.print("Accel: "); Serial.print(ax); Serial.print(" ");
  Serial.print(ay); Serial.print(" "); Serial.println(az);
  delay(500);
}

```

3. HW-837 - UV Light Sensor

```

#define UV_SENSOR_PIN A0

void setup() {
  Serial.begin(9600);
}

void loop() {
  int uvAnalog = analogRead(UV_SENSOR_PIN);
  float uvVoltage = uvAnalog * (5.0 / 1023.0);
  Serial.print("UV Analog Value: "); Serial.print(uvAnalog);
  Serial.print(" | Voltage: "); Serial.print(uvVoltage);
  Serial.println(" V");

  delay (1000);
}

```

This approach helped in identifying sensor issues and fine-tuning the system before connecting it to Blynk Cloud.

SOFTWARE IMPLEMENTATION:

To enable real-time monitoring and visualization of health parameters, we implemented Blynk as our IoT platform. Blynk allows seamless integration of hardware and cloud-based data monitoring, accessible via both desktop and mobile applications.

Steps Involved in Software Implementation:

1. Blynk Account Setup
 - Created a project in Blynk Cloud.

- Configured the dashboard with widgets to display data from sensors - temperature, heart rate, SpO₂, UV index, and step count.
2. Arduino IDE & Blynk Library Integration
 - Installed and included the Blynk and WiFi libraries in the Arduino IDE.
 - Wrote Arduino code to send sensor data to the Blynk Cloud using Wi-Fi.
 - Used virtual pins to map sensor readings to the Blynk dashboard.
 3. Connecting Sensors with Blynk
 - The MAX30100 (pulse oximeter), MPU6050 (gyroscope sensor), and HW-837 (UV sensor) were connected to the Arduino Uno.
 - The sensor data was processed and sent to the Blynk app for real-time monitoring.
 4. Mobile & Desktop Dashboard Implementation
 - On the Blynk Mobile App, interactive widgets were used to visualize the health parameters.
 - On Blynk Web Dashboard, a similar interface was created for remote monitoring.
 5. Testing and Calibration
 - The system was tested with different conditions to ensure accuracy.
 - Debugging was done using serial monitor and Blynk event logs.

CODE:

```
#define BLYNK_TEMPLATE_ID "TMPL5MLg7FYCV"
#define BLYNK_TEMPLATE_NAME "Wearable Health Monitoring System"
#define BLYNK_AUTH_TOKEN "f73-ql388LuuPMsJ_AYZ7RJx4ZOdVrv"
#include <Wire.h>
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include "MAX30100_PulseOximeter.h"
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <NTPClient.h>
#include <WiFiUdp.h>

// ESP32 I2C Configuration
#define I2C_SDA 21
#define I2C_SCL 22
#define REPORTING_PERIOD_MS 1000
// WiFi Credentials
#define WIFI_SSID "Galaxy"
#define WIFI_PASSWORD "1dominos1"
```

```

// UV Sensor Pin
#define UV_SENSOR_PIN 36
// Sensor Objects
Adafruit_MPU6050 mpu;
PulseOximeter pox;

// NTP Client
WiFiUDP ntpUDP;
NTPClient timeClient(ntpUDP, "pool.ntp.org", 19800);

// Variables
int stepCount = 0;
float stepThreshold = 1.2;
float lastAccelZ = 0;
unsigned long lastStepTime = 0;
unsigned long debounceDelay = 250;
unsigned long lastUploadTime = 0;
unsigned long uploadInterval = 15000;
uint32_t tsLastReport = 0;

void onBeatDetected() {
  Serial.println("♥ Heartbeat detected!");
}

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);

  Wire.begin(I2C_SDA, I2C_SCL, 400000);

  if (!pox.begin()) {
    Serial.println("✖ MAX30100 NOT found!");
    while(1);
  }
  pox.setIRLedCurrent(MAX30100_LED_CURR_7_6MA);
  pox.setOnBeatDetectedCallback(onBeatDetected);

  if (!mpu.begin()) {
    Serial.println("✖ MPU6050 NOT found!");
    while(1);
  }
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
}

```



```

Serial.println("\n WiFi Connected!");
Blynk.config(BLYNK_AUTH_TOKEN);

timeClient.begin();
timeClient.update();
}

void loop() {
  Blynk.run();
  pox.update();

  timeClient.update();
  int currentHour = timeClient.getHours();

  sensors_event_t accel, gyro, temp;
  mpu.getEvent(&accel, &gyro, &temp);
  float currentAccelZ = accel.acceleration.z;

  float uvIndex = 0;
  if (currentHour >= 6 && currentHour < 18) {
    uvIndex = analogRead(UV_SENSOR_PIN) * (3.3 / 4095.0);
    uvIndex = constrain(map(uvIndex, 0.0, 1.0, 0.0, 15.0), 0, 15);
  }

  if (abs(currentAccelZ - lastAccelZ) > stepThreshold &&
      (millis() - lastStepTime > debounceDelay)) {
    stepCount++;
    lastStepTime = millis();
  }
  lastAccelZ = currentAccelZ;

  if (millis() - tsLastReport > REPORTING_PERIOD_MS) {
    Serial.print("Heart Rate: ");
    Serial.print(pox.getHeartRate());
    Serial.print("bpm | SpO2: ");
    Serial.print(pox.getSpO2());
    Serial.println("%");
    tsLastReport = millis();
  }

  if (millis() - lastUploadTime > uploadInterval) {
    // Send data to Blynk
    Blynk.virtualWrite(V0, stepCount);    // Step count
    Blynk.virtualWrite(V1, temp.temperature); // Temperature
    Blynk.virtualWrite(V2, uvIndex);      // UV Index
    Blynk.virtualWrite(V3, pox.getHeartRate()); // Pulse rate
    Blynk.virtualWrite(V4, pox.getSpO2()); // SpO2
    lastUploadTime = millis();
  }
}

```

WORKING:

1. System Initialization

- When the device is powered on, the ESP32 microcontroller initializes all necessary peripherals, including Wi-Fi, I2C (for sensor communication), and Blynk IoT connectivity.
- The Wi-Fi module attempts to connect to a pre-configured network. If the connection is not established, the ESP32 keeps retrying until successful.
- The NTP (Network Time Protocol) client fetches the current time from an online server. This is used to manage time-sensitive tasks like UV sensor activation (6 AM - 6 PM).

2. Sensor Data Collection

Once the ESP32 system boots and all sensors are initialized, data collection begins through various sensor interfaces:

- **MAX30100 Pulse Oximeter**
This sensor emits infrared and red light through an LED placed on one side of the sensor. When a finger is placed over it, the light passes through the tissue and is received by a photodetector on the other side. The variation in the received light is directly related to the blood flow, allowing it to capture pulse rate and SpO₂ levels.
- **MPU6050 Accelerometer & Gyroscope**
The accelerometer inside the MPU6050 captures movement data in X, Y, and Z axes. The Z-axis acceleration is specifically used to detect steps based on sudden changes that cross a defined threshold. Though it also has a gyroscope, this project mainly focuses on step counting using acceleration data.
- **HW-837 UV Sensor**
This analog sensor detects the intensity of UV light in the environment. It is connected to one of the ESP32's analog pins, which reads varying voltage levels. These levels are proportional to the UV radiation and are later mapped to a UV index value (0–15). The sensor is programmed to be active only between 6 AM and 6 PM, using real-time data from an NTP time server.

3. Data Processing

After collecting raw values from all the connected sensors, the ESP32 processes the data to convert it into meaningful health metrics that are easy to interpret. Here's how the processing happens for each sensor:

MAX30100 Pulse Oximeter:

- The photodetector in the MAX30100 sensor captures light reflected from body tissues after the emission of red and infrared (IR) light by built-in LEDs.

- These raw signals are processed using the MAX30100_PulseOximeter library. The library handles signal filtering and peak detection to extract accurate data.
- Specifically, the functions used are:
 - `pox.getHeartRate()` – returns the Heart Rate in beats per minute (bpm).
 - `pox.getSpO2()` – returns the Oxygen Saturation (SpO₂) as a percentage (%).

MPU6050 – Motion, Step Count & Temperature Measurement

The MPU6050 sensor captures 3-axis accelerometer and gyroscope data to detect movement and orientation. In this project, only the accelerometer (Z-axis) is used to count steps.

- **Step Detection**

The ESP32 checks the change in Z-axis acceleration (`accel.acceleration.z`) between readings. If the change exceeds a threshold of 1.2, and at least 250 milliseconds have passed since the last detected step (debounce), a new step is counted.

- **Debounce Filtering**

The 250 ms debounce time (`debounceDelay = 250`) prevents false step counts due to noise or minor movements. This ensures that only real steps are registered.

- **Temperature Measurement**

The MPU6050 also includes an internal temperature sensor. Temperature is retrieved using the `mpu.getEvent()` function and accessed via `temp.temperature`. This temperature value is sent to Blynk every 15 seconds for monitoring body/environmental heat levels.

HW-837 – UV Exposure Sensor:

- The HW-837 analog UV sensor reads UV intensity using a photodiode that responds to UV-A and UV-B rays.
- The analog signal is read using `analogRead(UV_SENSOR_PIN)`, converted to voltage, and mapped to the UV Index range (0–15).
- To conserve power and ensure accurate results, UV sensing is active only between 6 AM to 6 PM, based on the time fetched from NTPClient.

4. Wireless Data Transmission

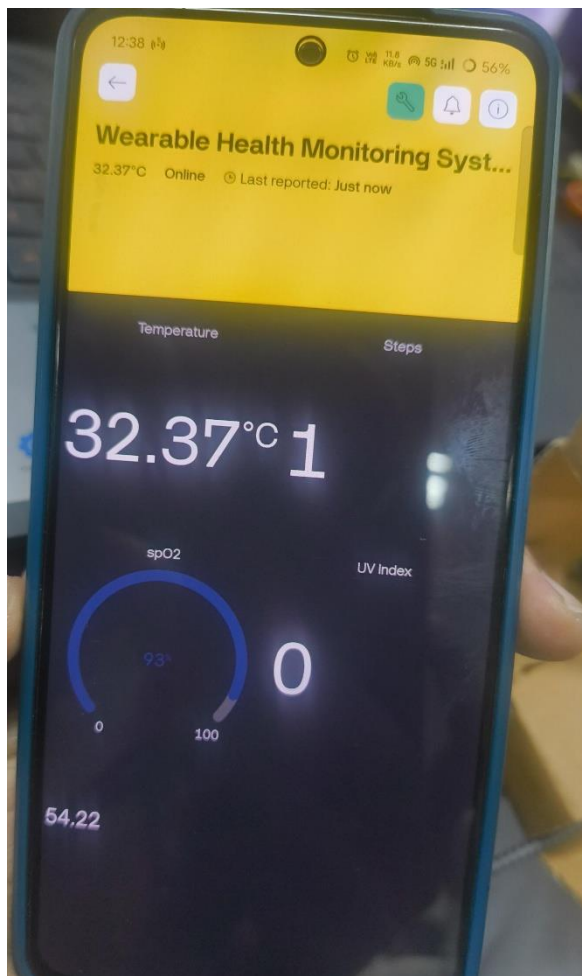
- After processing the sensor readings, the ESP32 transmits the data wirelessly to the Blynk IoT platform over Wi-Fi using the `Blynk.virtualWrite()` function.
- Each sensor output is mapped to a specific virtual pin, which acts as a channel between the ESP32 and Blynk app/dashboard:
 - V0 → Step Count (from MPU6050)
 - V1 → Body Temperature (from DS18B20)
 - V2 → UV Index (from HW-837)
 - V3 → Heart Rate in bpm (from MAX30100)
 - V4 → SpO₂ in % (from MAX30100)
- The data is updated every 15 seconds, controlled using the variable `uploadInterval` in the code. This ensures efficient bandwidth usage while maintaining near real-time monitoring.

- These virtual pins are connected to widgets in the Blynk mobile and web dashboards, allowing users to easily visualize their health data from anywhere.
- This wireless transmission eliminates the need for wired connections or manual data collection, improving portability and user-friendliness.

5.Power Management

- Sensor Activation Based on Need:
 - The MAX30100 Pulse Oximeter operates only when needed for pulse rate and SpO₂ measurement, avoiding continuous power consumption.
 - The MPU6050 Motion Sensor is read at regular intervals for step detection instead of being active all the time.
 - The HW-837 UV Sensor is enabled only between 6 AM and 6 PM, using real-time data from NTPClient to conserve energy.
- Battery-Powered Operation:
 - The device is powered by a 3.7V Li-ion battery, ensuring portability.
 - The ESP32 manages power distribution efficiently to keep the system running smoothly without excessive drain.
- Controlled Data Transmission:
 - The ESP32 sends sensor readings to Blynk every 15 seconds, reducing unnecessary Wi-Fi usage and saving power.

OUTPUT:



```

18 #define WIFI_SSID "Vignesh"
19 #define WIFI_PASSWORD "90754321"
20 #define REPORTING_PERIOD 15000
21 Adafruit MPU6050 mpw;
22 PulseOximeter pox;
23 WiFiUDP ntpudp;
24 NTPClient timeClient(ntpudp, "pool.ntp.org", 13000);
25 BlynkTimer timer;
26
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
[ 46922][E][Wire.cpp:489] requestFrom(): I2C read bus error: 4
[ 46929][E][Wire.cpp:489] requestFrom(): I2C read bus error: 4
[ 46936][E][Wire.cpp:489] requestFrom(): I2C read bus error: 4
[ 46944][E][Wire.cpp:489] requestFrom(): I2C read bus error: 4
[ 46951][E][Wire.cpp:489] requestFrom(): I2C read bus error: 4
[ 48327][E][Wire.cpp:489] requestFrom(): I2C read bus error: 4
[ 48335][E][Wire.cpp:489] requestFrom(): I2C read bus error: 4
[ 48342][E][Wire.cpp:489] requestFrom(): I2C read bus error: 4
[ 48386][E][Wire.cpp:489] requestFrom(): I2C read bus error: 4
Heartbeat detected!

```

ADVANTAGES:

- **Multi-Parameter Tracking:** Continuously monitors pulse rate, SpO₂, body temperature, motion, and UV exposure in a compact wearable device.
- **Proactive Health Alerts:** Notifies users of abnormal pulse rate variations and provides UV exposure warnings between 6 AM and 6 PM to encourage preventive actions.
- **Real-Time Data Monitoring:** Displays live health data on the Blynk IoT app, accessible from both mobile and desktop platforms.
- **Portable & Rechargeable:** Powered by a rechargeable battery, ensuring continuous operation without reliance on external power sources.
- **User-Friendly Interface:** Simple setup and intuitive dashboard make it easy for users to interpret health data and receive alerts.
- **Wireless & Remote Access:** Uses Wi-Fi connectivity to send data to the cloud, allowing users to check their health status remotely.
- **Affordable & Expandable:** Cost-effective compared to advanced medical wearables, with the flexibility to integrate additional sensors in the future.

FUTURE SCOPE:

The current wearable health monitoring system is a strong prototype for real-time health tracking. However, there is significant potential to enhance its design, functionality, and user experience in the future. Some of the key improvements include:

- **Custom PCB Design**

The current prototype is built using a breadboard and jumper wires, which makes the device bulky and less reliable for long-term or wearable use. In future development, this setup can be replaced by a custom-designed PCB (Printed Circuit Board) that integrates all components in a compact and organized layout. This will reduce the device size, enhance durability, and make it suitable for wearable formats such as smart rings, wristbands, or health-monitoring badges.
- **Additional Health Sensors**

To improve the system's medical capabilities, various advanced sensors can be added. These include:

 - An ECG sensor for monitoring heart rhythm and detecting irregularities.
 - A GSR (Galvanic Skin Response) sensor to assess emotional states and stress levels.
 - A blood pressure sensor for tracking cardiovascular health.
 - Advanced motion sensors or posture detection modules for identifying falls or improper sitting/standing posture, especially useful for elderly care.
- **Cloud Storage & Doctor Integration**

The device can be connected to cloud platforms such as Firebase or AWS IoT,

allowing for secure and long-term storage of health data. This enables remote access to health metrics, making it easier for doctors or caregivers to monitor a patient's condition in real-time. Such integration supports early diagnosis, continuous monitoring, and better health management without the need for frequent hospital visits.

CONCLUSION:

The wearable health monitoring system developed in this project successfully integrates multiple biomedical sensors, including heart rate, SpO₂ (MAX30100), motion (MPU6050), and UV exposure, into a compact, Arduino-based platform. By continuously collecting and transmitting real-time health data to Blynk IOT, this device empowers users and healthcare professionals to monitor vital signs remotely. The system demonstrates significant potential for early detection of health anomalies, especially for elderly and high-risk patients. Future enhancements such as miniaturization, battery optimization, and machine learning-based anomaly detection can further improve its utility in telemedicine and personal health tracking.